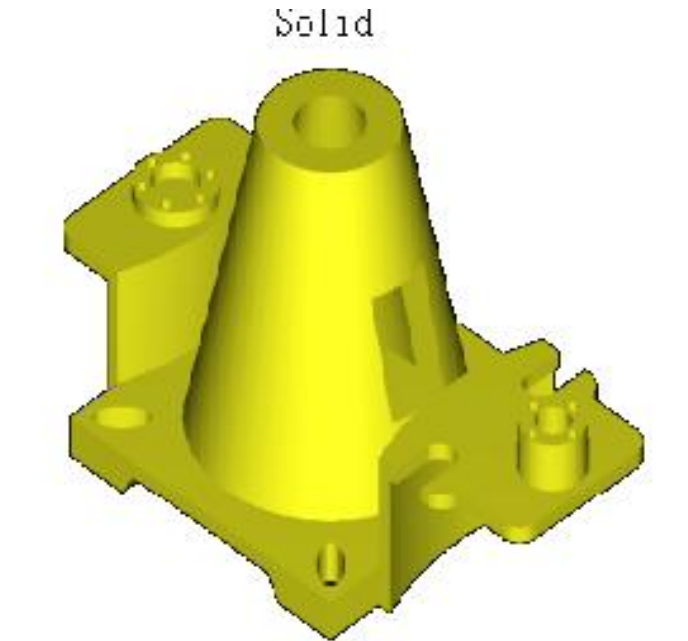
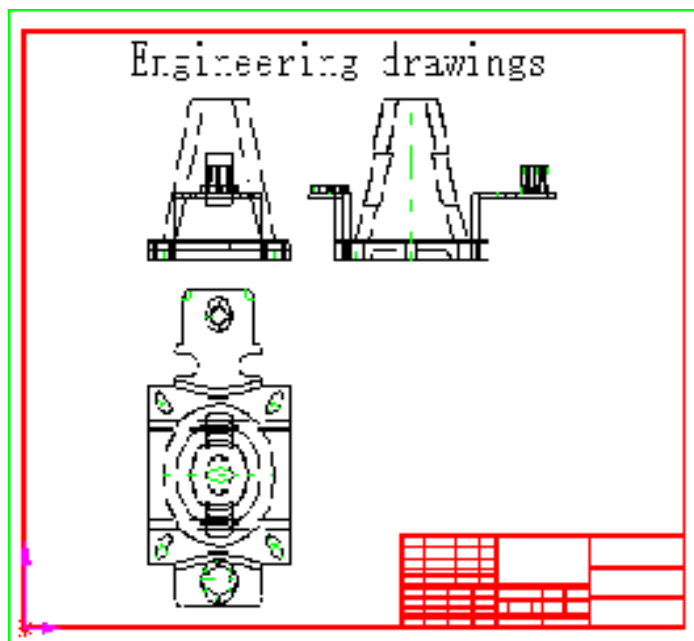
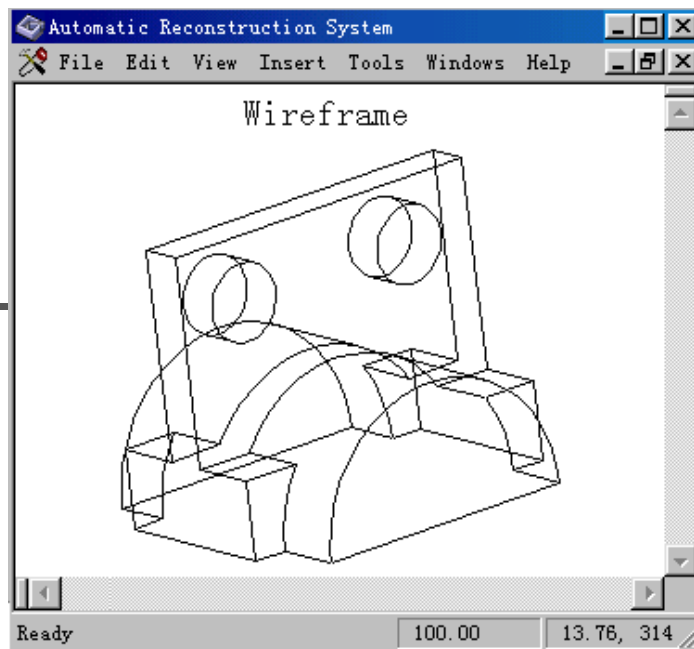


# 第四章 二维图形的生成技术

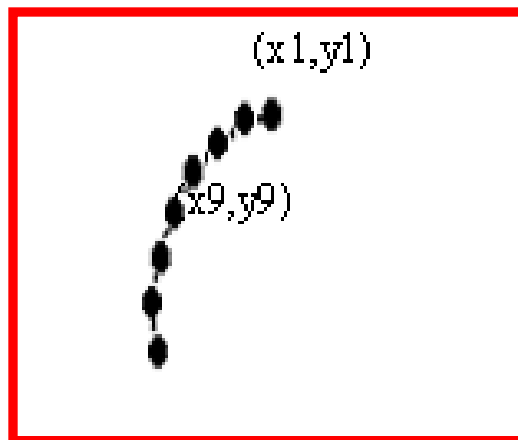
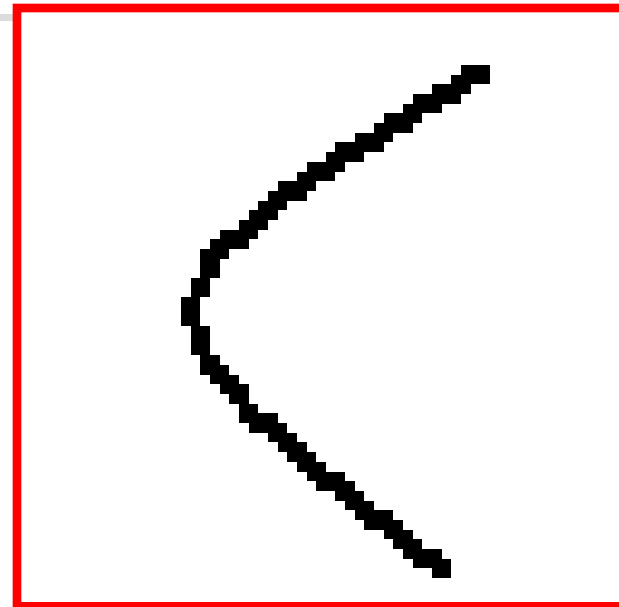
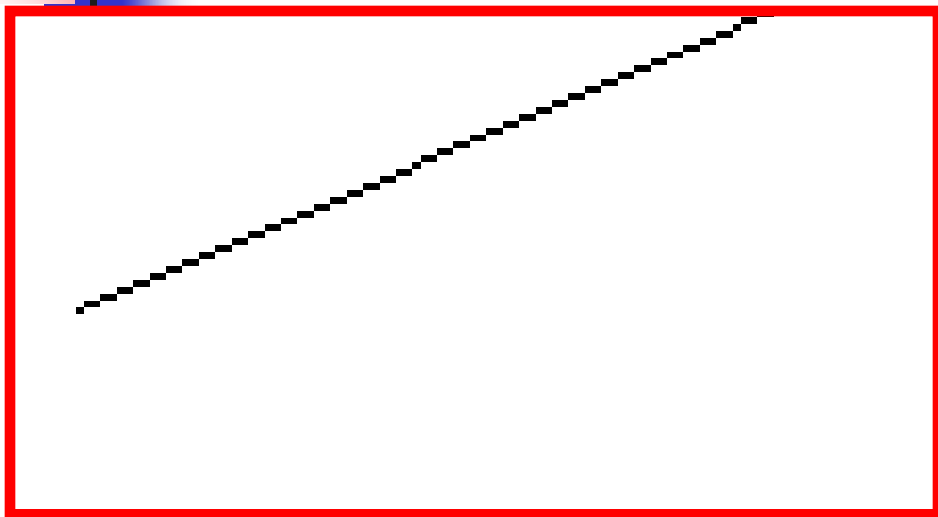
---

谢忠红  
南京农业大学

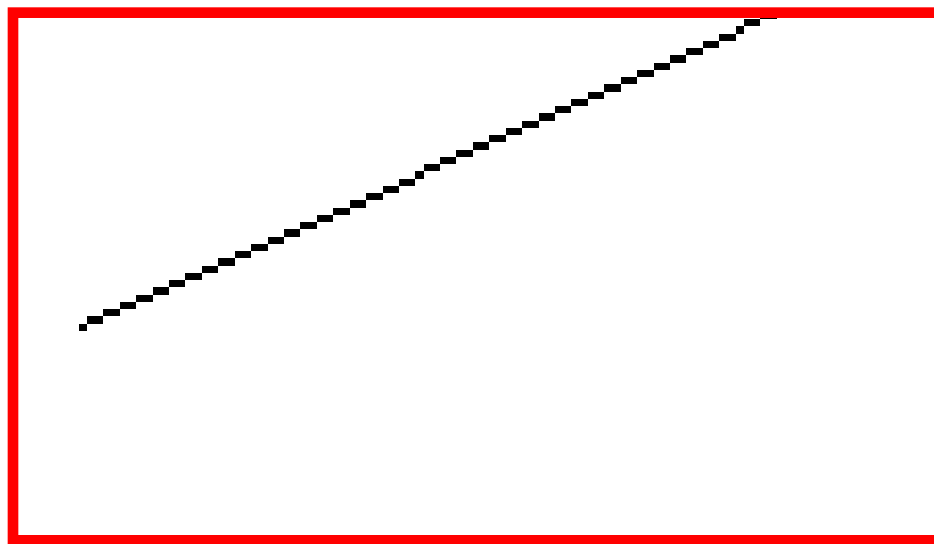
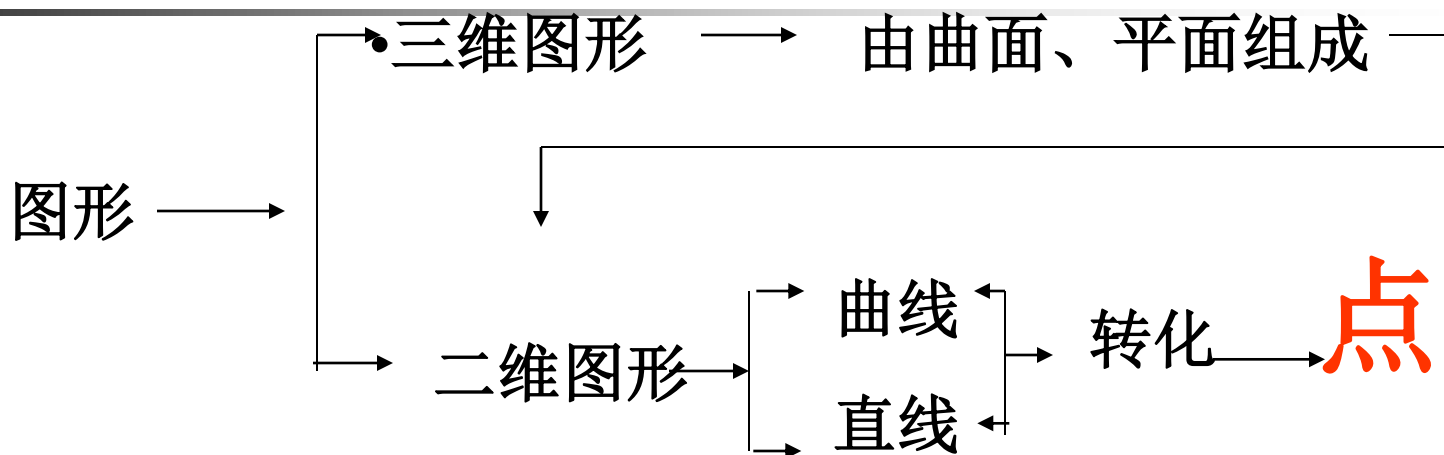
# 复杂图形



# 直线曲线都是点的集合



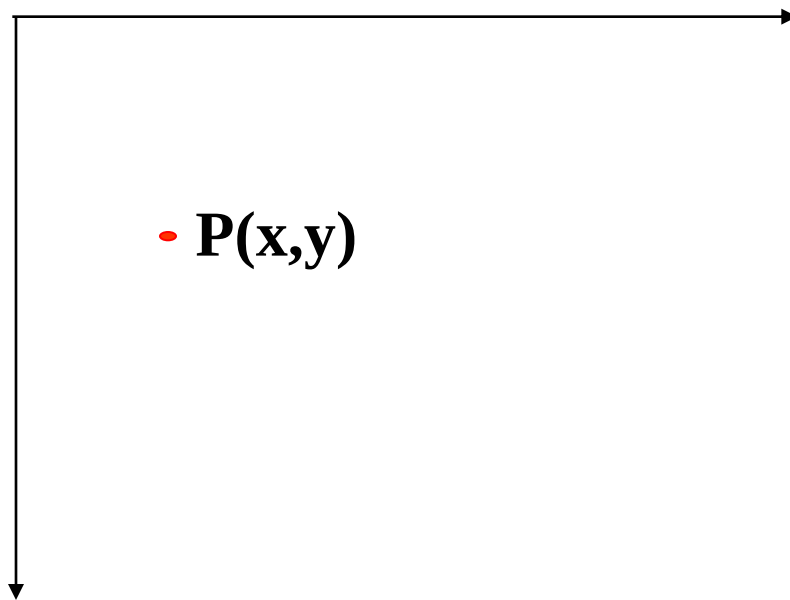
## 4.1 基本图元素-点



- **点**是图形中最基本的图素，直线、曲线以及其它图元都是点的集合。

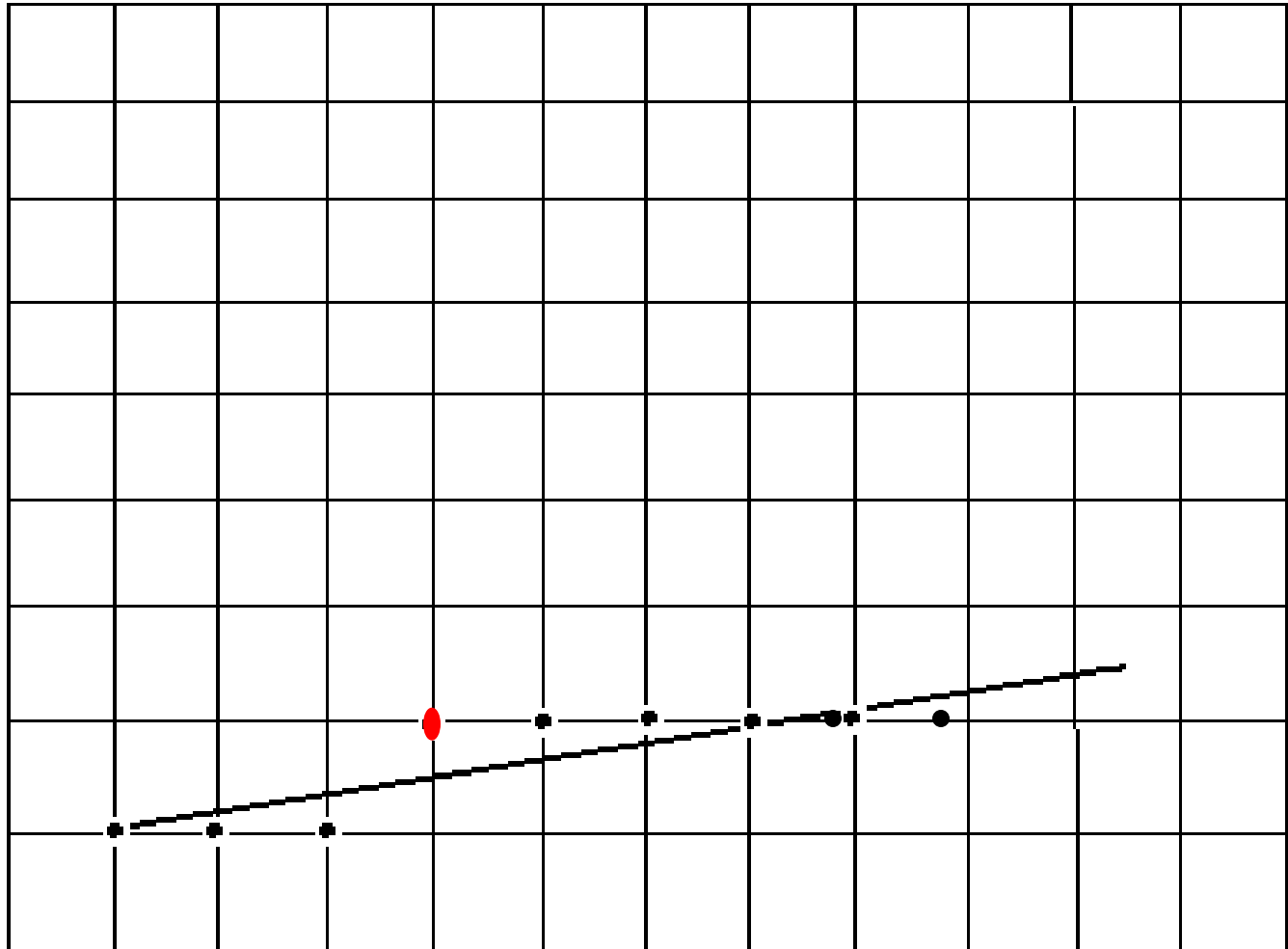
**putpixel(x,y,color)**

- **函数作用：**使屏幕上坐标**(x,y)**的**点**辉亮



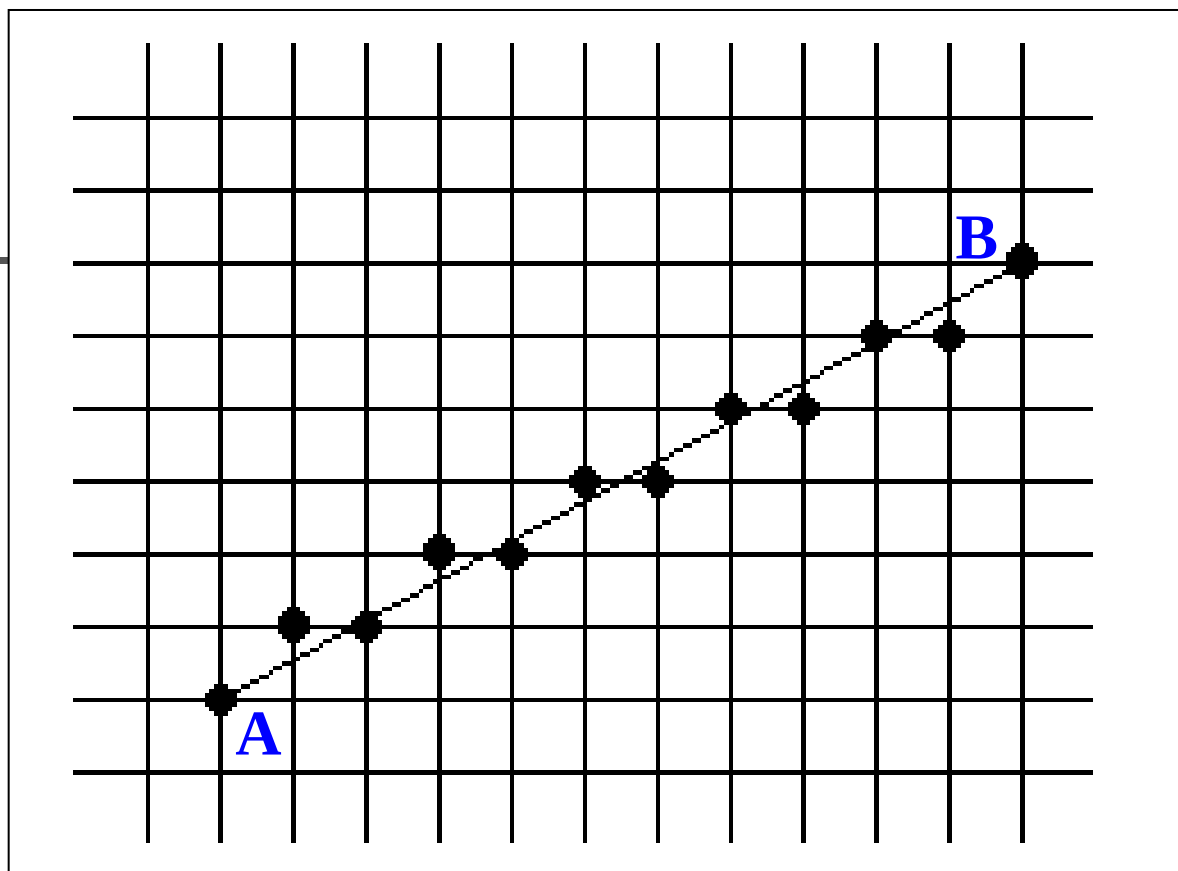
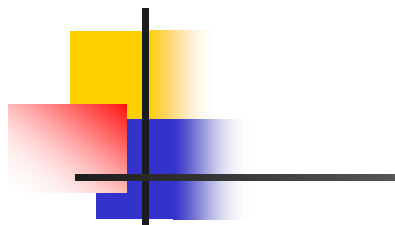


- Putpixel (5 , 7 , RED)



**直线**是点的集合，（几何学定义：直线被定义为两个点之间的最小距离）

- **图形学的直线：**
- **（1）** 图形显示器是由一个个排列有序的像素点构成的，**像素之间为均匀的网格**
- **（2）** 点的坐标是指点所在像素点的位置，而像素点的位置是整型的，**画一条直线实际上是依据一系列计算出来的位置得到靠近该位置的像素点。**

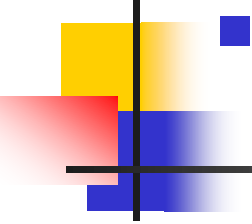


对于光栅显示器来说:

- 1、像素间为均匀网格
- 2、整型坐标系

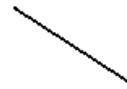
$(10.48, 20.51) \rightarrow (10, 21)$



- 
- 后果：对坐标取整使得显示的线段具有锯齿现象
  - 改进的方法：提高屏幕的分辨率
- 



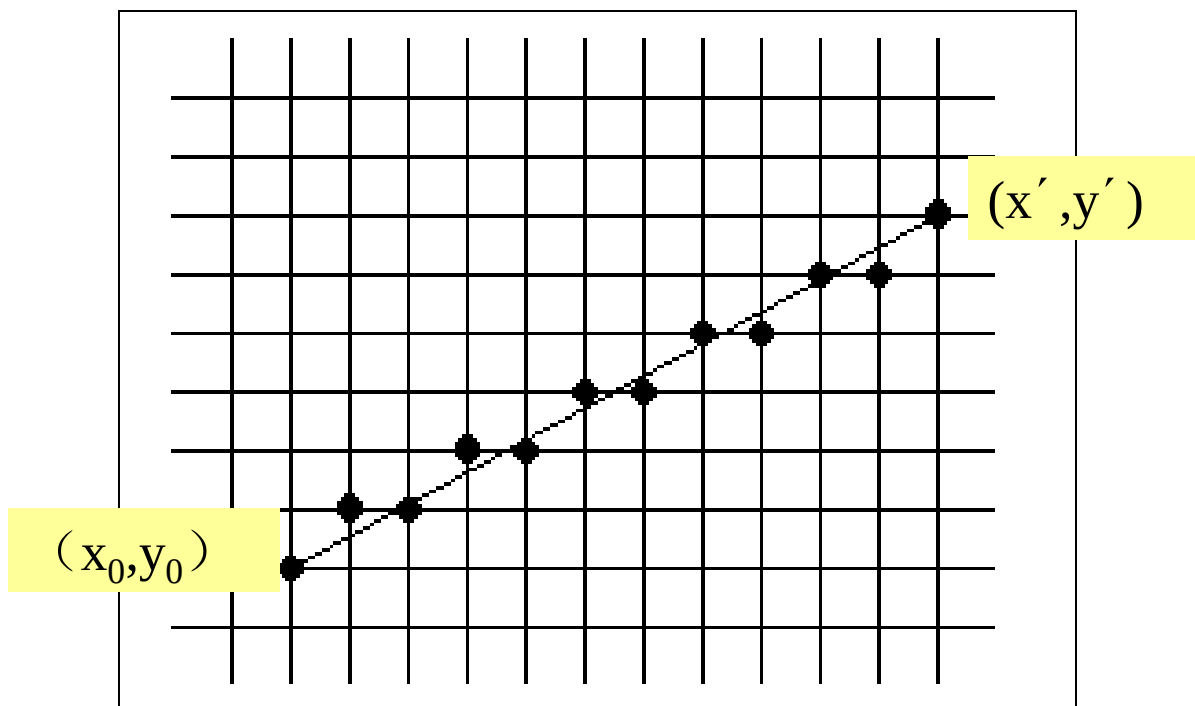
**800\*600**

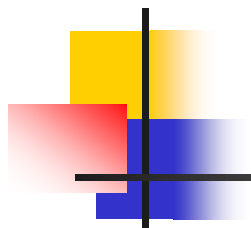


**1024\*1024**

## 4.2 直线段的生成

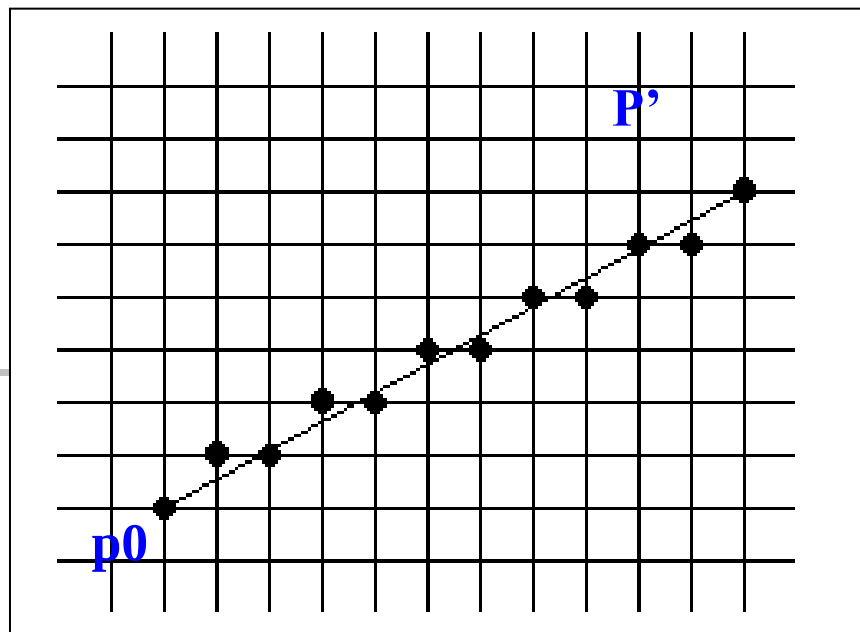
- 1. 数值微分法 (DDA) 基本思想:
- 确定最佳逼近该直线的一组象素，并按扫描线的顺序显示这组象素点。





■ 已知条件:

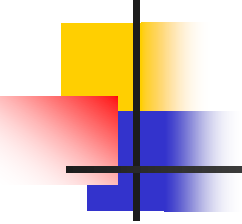
待连线两点:



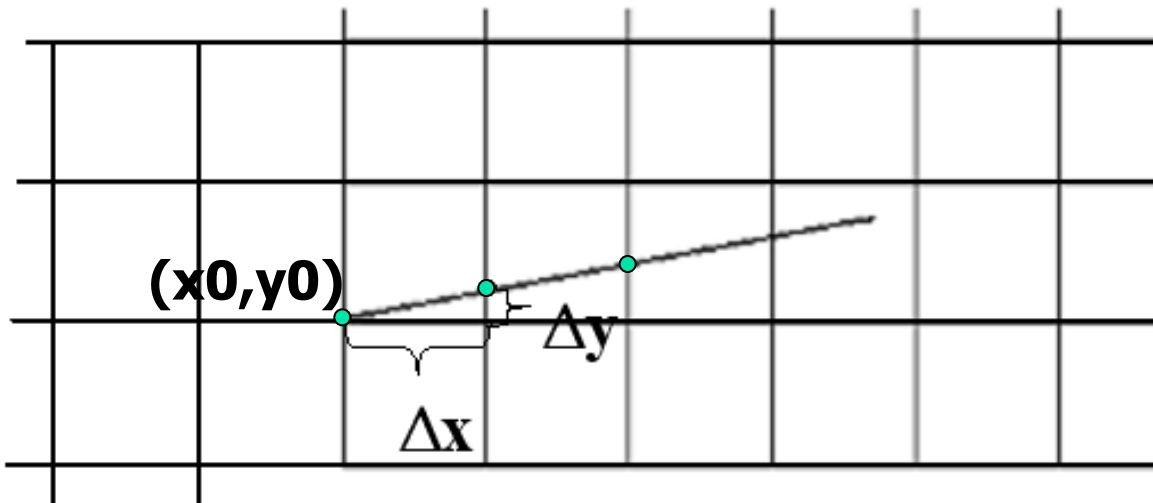
$$P_0 (x_0, y_0) \quad , \quad P' (x', y')$$

直线斜率为:

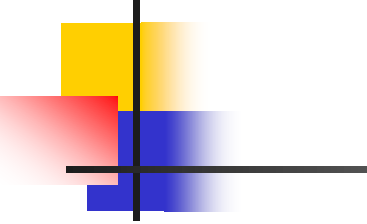
$$k = \frac{y' - y_0}{x' - x_0}$$


$$K = \Delta y / \Delta x$$

$$\Delta y = k \Delta x$$



$$\begin{aligned} x_1 &= x_0 + \Delta x, & x_2 &= x_1 + \Delta x, & \dots, & x_i &= x_{i-1} + \Delta x \\ y_1 &= y_0 + k \Delta x, & y_2 &= y_1 + k \Delta x, & \dots, & y_i &= y_{i-1} + k \Delta x \end{aligned}$$


$$x_i = x_{i-1} + \Delta x$$
$$y_i = y_{i-1} + k \Delta x$$

设:  $\Delta x = 1$

$$x_i = x_{i-1} + 1$$
$$y_i = y_{i-1} + k \quad (k \leq 1)$$

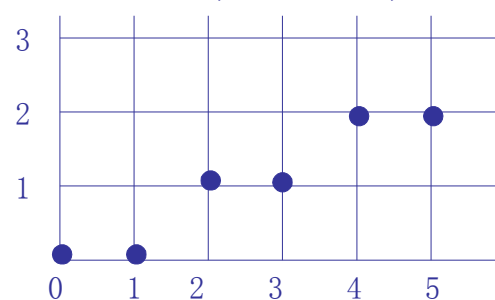
点的坐标: (  $x_i$ ,  $\text{round}(y_i)$  )

•即: 当 $x$ 每递增1,  $y$ 递增 $k$ (即直线斜率);

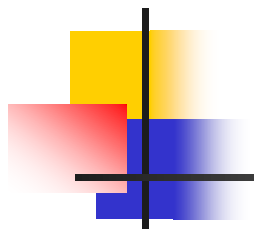
例：用DDA算法画直线段 $P_0(0, 0) \sim P_1(5, 2)$

$$k = (2 - 0) / (5 - 0) = 0.4$$

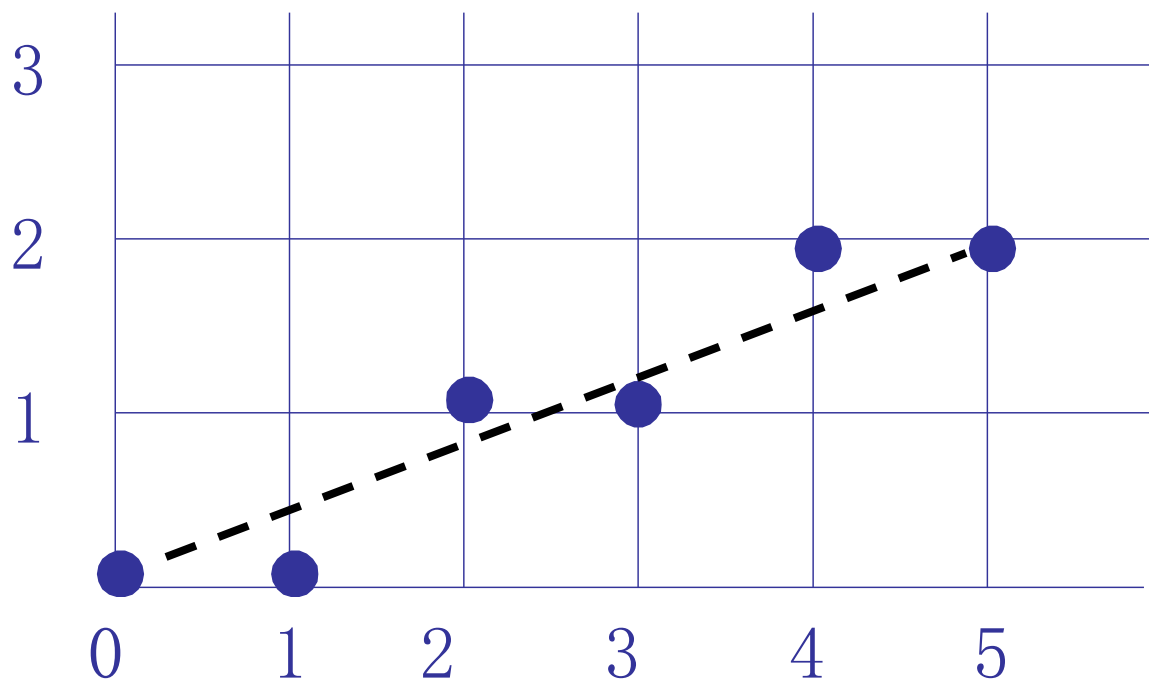
Line:  $P_0(0, 0) \sim P_1(5, 2)$

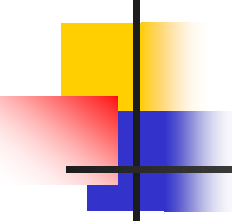


| $i$ | $X_i = x_{i-1} + 1$ | $y_i = y_{i-1} + k$ | $\text{round}(y_i)$     | 坐标    |
|-----|---------------------|---------------------|-------------------------|-------|
| 0   | 0                   | 0                   | $\text{round}(0) = 0$   | (0,0) |
| 1   | 1                   | $0 + 0.4 = 0.4$     | $\text{round}(0.4) = 0$ | (1,0) |
| 2   | 2                   | $0.4 + 0.4 = 0.8$   | $\text{round}(0.8) = 1$ | (2,1) |
| 3   | 3                   | $0.8 + 0.4 = 1.2$   | $\text{round}(1.2) = 1$ | (3,1) |
| 4   | 4                   | $1.2 + 0.4 = 1.6$   | $\text{round}(1.6) = 2$ | (4,2) |
| 5   | 5                   | $1.6 + 0.4 = 2.0$   | $\text{round}(2.0) = 2$ | (5,2) |

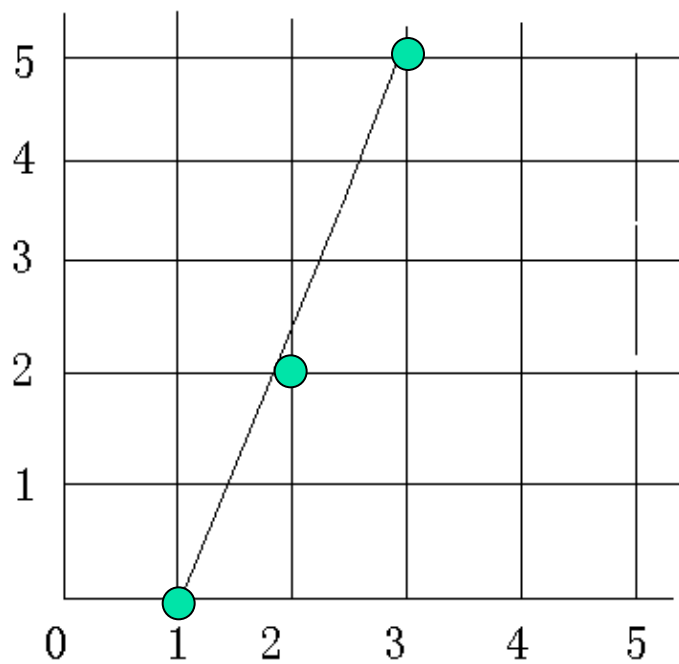
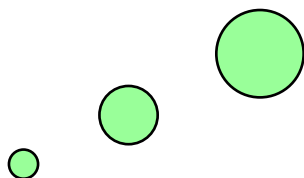


Line:  $P_0(0, 0) \text{--} P_1(5, 2)$



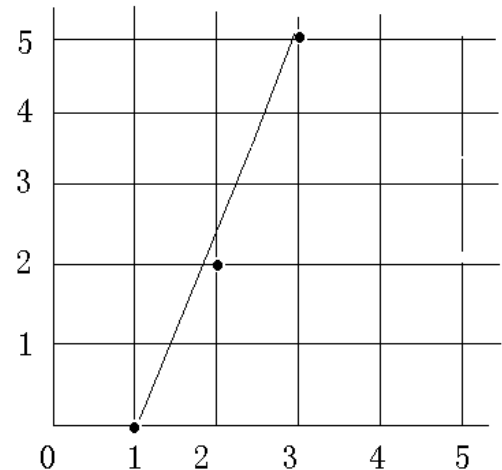
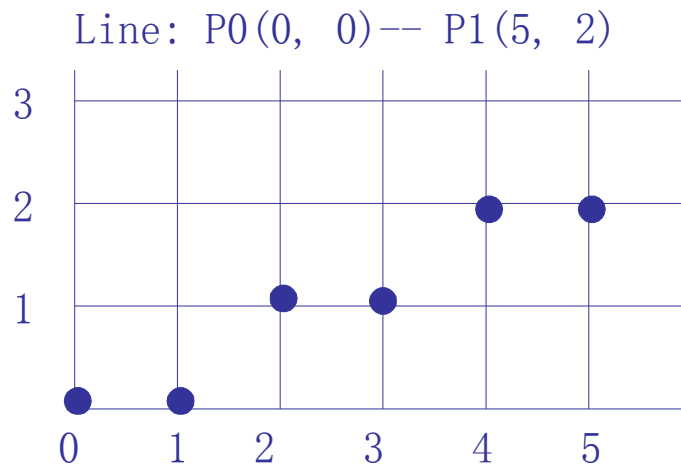


该方法能画出  
所有直线吗？





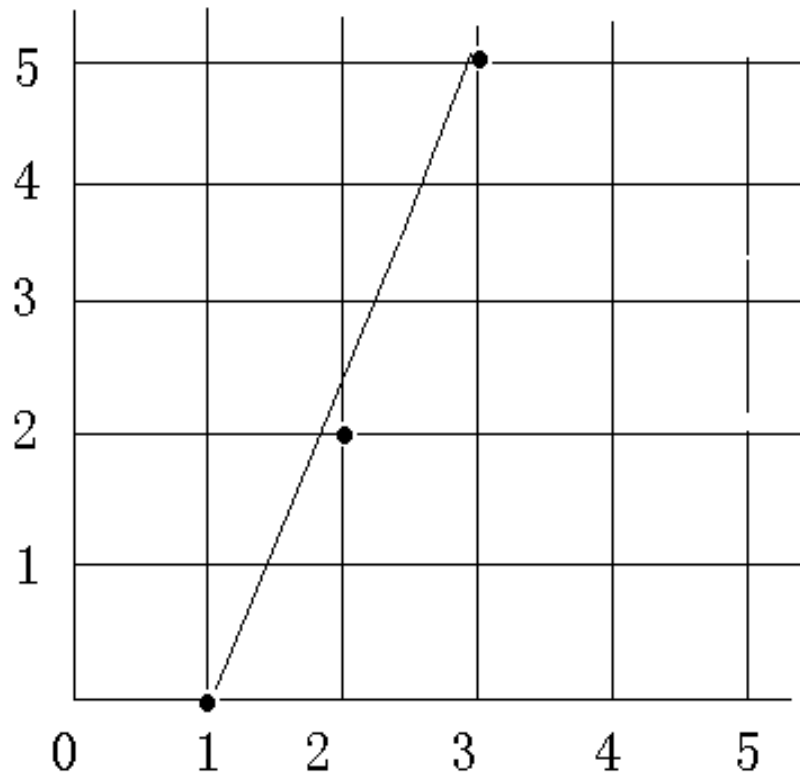
- 分析:
- 上述分析的算法仅适用于 $|k| \leq 1$ 的情形。
- 原因:
- 当 $|k| \leq 1$ 时,  $x$ 每增加1( $\Delta x=1$ ),  $y$ 最多增加1( $|\Delta y| = |k| \Delta x \leq 1$ )。如果,
- 当 $|k| > 1$ 时,  $x$ 每增加1,  $y$ 可能会增加大于1的数( $|\Delta y| = |k| \Delta x > 1$ ), 这时像素点会稀疏, 所画直线会断断续续。

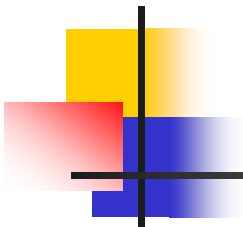


**K>1**时使用如下公式的效果图

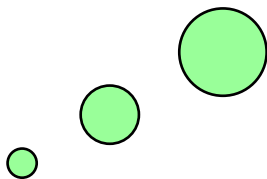
$$x_i = x_{i-1} + 1$$

$$y_i = y_{i-1} + k \quad (k > 1)$$





**K>1时怎么办?**



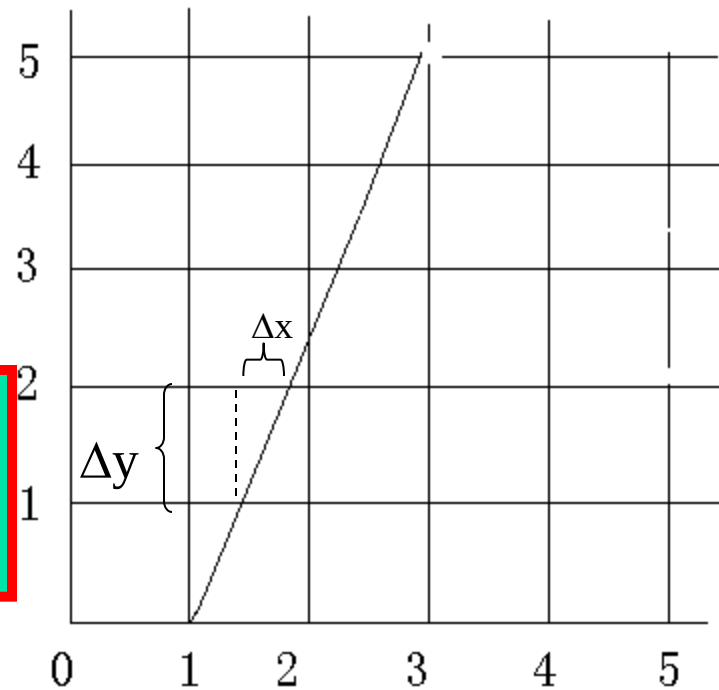
已知直线  $y=kx+b$

$|k|>1$

$$K = \Delta y / \Delta x$$

$$\text{令} \begin{cases} \Delta y = 1 \\ \Delta x = \Delta y * 1/k \end{cases}$$

$$\text{即:} \begin{cases} y_i = y_{i-1} + 1 \\ x_i = x_{i-1} + 1/k \end{cases}$$



( round (xi), yi )

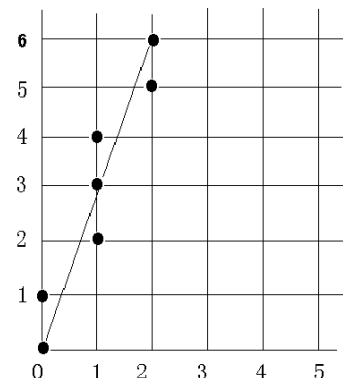
- 例：画出 $p_0(0,0)$ 和 $p'(2,6)$ 之间的一条直线。

$$K=(6-0)/(2-0)=3>1$$

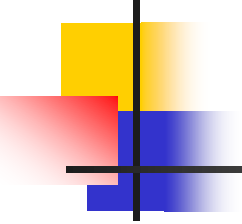
- 采用通过 $y_i$ 的变化确定 $x_i$ 的变化

$$\text{即： } \Delta y = 1 \quad \Delta x = 1/k$$

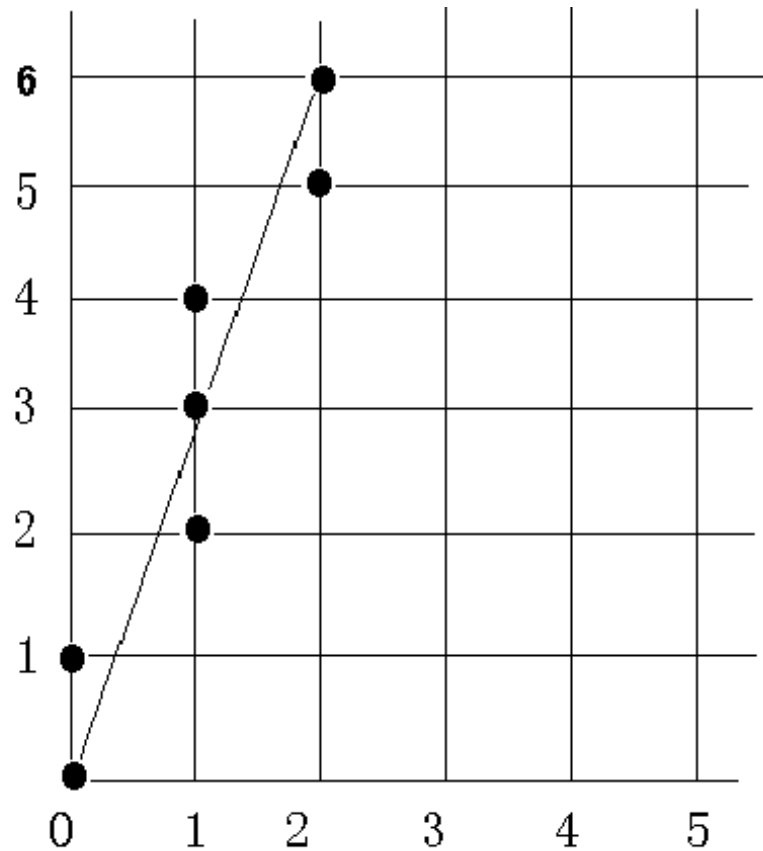
$$1/k = 1/3 = 0.33$$

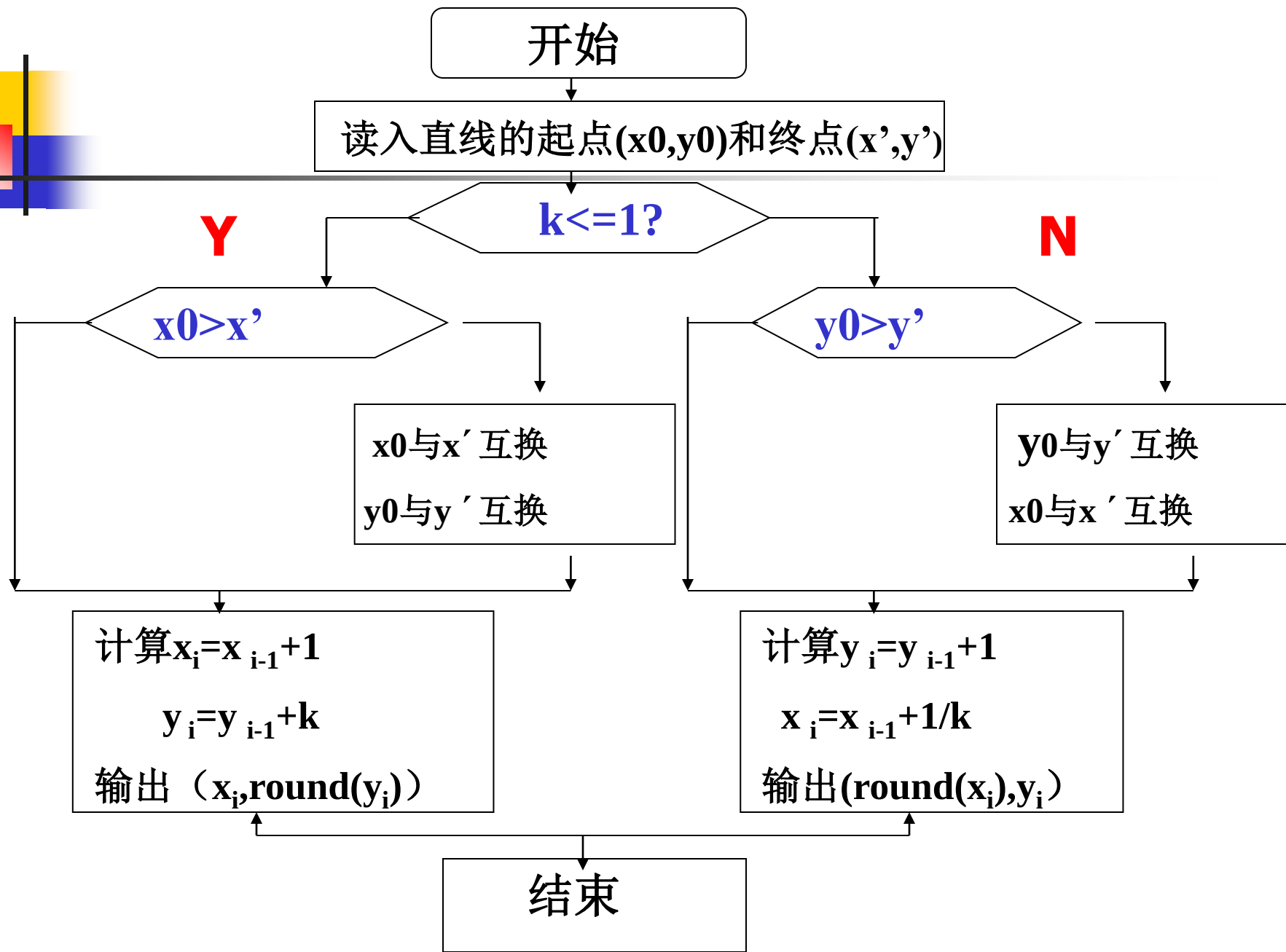
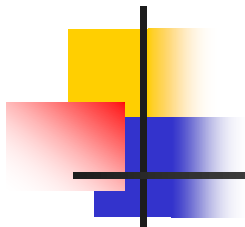


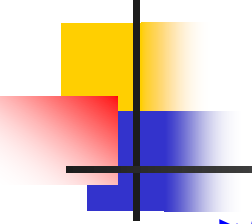
| $i$ | $y_i = y_{i-1} + 1$ | $x_i = x_{i-1} + 1/k$ | $\text{round}(x_i)$      | 坐标    |
|-----|---------------------|-----------------------|--------------------------|-------|
| 0   | 0                   | 0                     | $\text{round}(0) = 0$    | (0,0) |
| 1   | 1                   | $0 + 0.33 = 0.33$     | $\text{round}(0.33) = 0$ | (0,1) |
| 2   | 2                   | $0.33 + 0.33 = 0.66$  | $\text{round}(0.66) = 1$ | (1,2) |
| 3   | 3                   | $0.66 + 0.33 = 0.99$  | $\text{round}(0.99) = 1$ | (1,3) |
| 4   | 4                   | $0.99 + 0.33 = 1.32$  | $\text{round}(1.32) = 1$ | (1,4) |
| 5   | 5                   | $1.32 + 0.33 = 1.65$  | $\text{round}(1.65) = 2$ | (2,5) |
| 6   | 6                   | $1.65 + 0.33 = 1.98$  | $\text{round}(1.98) = 2$ | (2,6) |



# Line(0 , 0 , 2 , 6)





- 
- 
- 数值微分(**DDA**)法缺点:
  - 在此算法中, **y**、**k**必须是**float**, 且每一步都必须对**y**进行舍入取整, 不利于硬件实现。

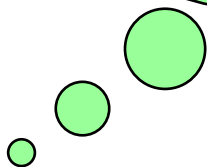




# DDA圆的生成算法

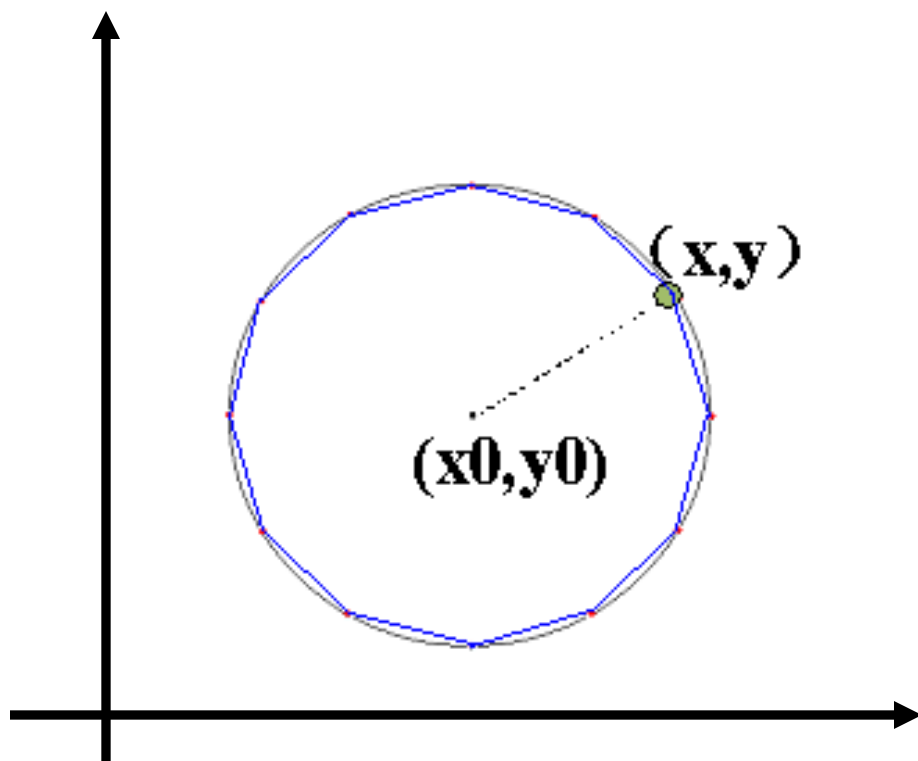
---

如何使用计算机生成  
一个圆或圆弧呢？



# 角度DDA法生成圆

**原理：** 把圆或者圆弧分成**N**等分。用**N**段相邻的直线来逼近圆弧或圆。



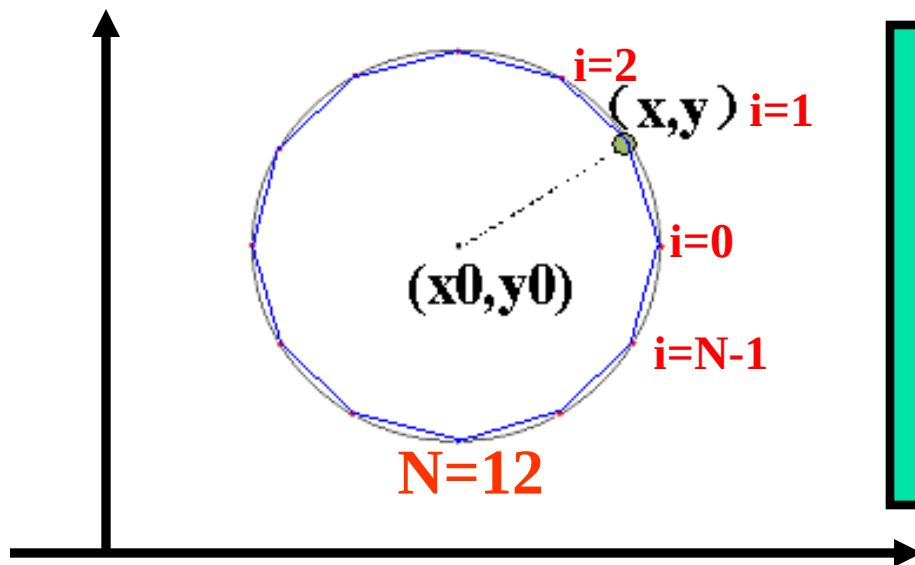
圆心在  $(x_0, y_0)$   
半径为  $R$

圆的参数方程

$$\begin{cases} x = x_0 + R\cos\theta \\ y = y_0 + R\sin\theta \end{cases}$$

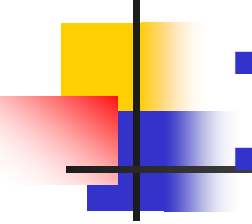
若将整个圆分成 **$N$** 等分,  **$\Delta\theta=2\pi/N$** ,  
则有参数式递推公式:

- 其中 **$\theta_i = i * \Delta\theta$**   **$i=0,1,2,3,\dots,N-1$**



$$\begin{cases} X_i = x_0 + R \cos \theta_i \\ Y_i = y_0 + R \sin \theta_i \end{cases}$$

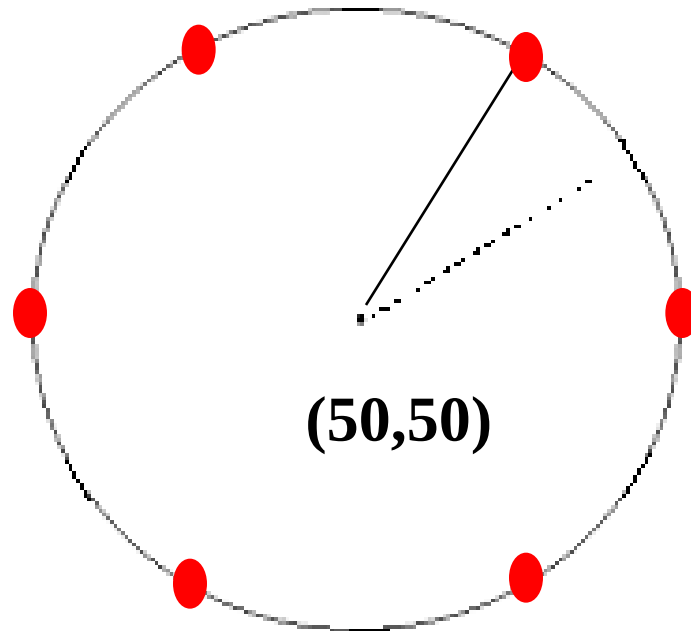
- 最后将相邻的  $(x_i, y_i)$   $(x_{i+1}, y_{i+1})$  用直线段一一相连就可近似地画出圆了。



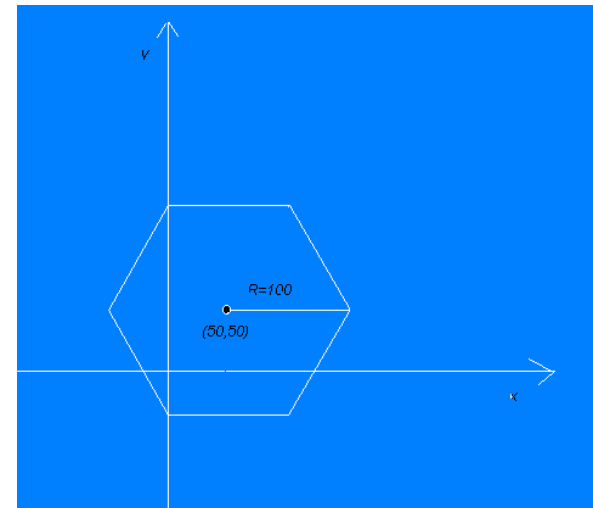
■ 例：圆心 $(x_0, y_0) = (50, 50)$

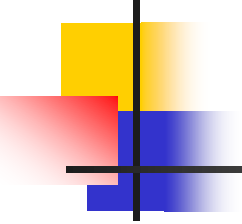
■  $R=100$   $N=6$

■  $\Delta\theta = 2\pi/6 = \pi/3$



- $x_0 = 50 + 100\cos(0 \cdot \pi/3)$   
 $y_0 = 50 + 100\sin(0 \cdot \pi/3)$
- $x_1 = 50 + 100\cos(1 \cdot \pi/3)$   
 $y_1 = 50 + 100\sin(1 \cdot \pi/3)$
- $x_2 = 50 + 100\cos(2 \cdot \pi/3)$   
 $y_2 = 50 + 100\sin(2 \cdot \pi/3)$
- $x_3 = 50 + 100\cos(3 \cdot \pi/3)$   
 $y_3 = 50 + 100\sin(3 \cdot \pi/3)$
- $x_4 = 50 + 100\cos(4 \cdot \pi/3)$   
 $y_4 = 50 + 100\sin(4 \cdot \pi/3)$
- $x_5 = 50 + 100\cos(5 \cdot \pi/3)$   
 $y_5 = 50 + 100\sin(5 \cdot \pi/3)$
- $x_6 = 50 + 100\cos(6 \cdot \pi/3)$   
 $y_6 = 50 + 100\sin(6 \cdot \pi/3)$
- [P DDA\_circle]





---

■ 请写出**DDA**画圆算法！

**void DDA\_circle(int n ,int x0,int y0,int radius)**

**// 把圆分成n等分, (x0,y0)为圆心坐标, radius为圆的半径**

**{int i;**

**int x\_start, y\_start ; //线段的起点坐标**

■ **Int x\_end,y\_end; //线段的中点坐标**

■ **for(i=0;i<n;i++)//循环地算初起点和终点坐标并连接这两点**

■ **{**

**x\_start=x0+radius\*cos(i\*2\*3.14/n) ;**

■ **y\_start=y0+radius\*sin(i\*2\*3.14/n) ;**

**x\_end=x0+radius\*cos((i+1)\*2\*3.14/n) ;**

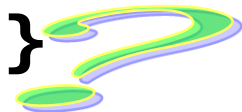
**y\_end=y0+radius\*sin((i+1)\*2\*3.14/n) ;**

**my\_line(x\_start,y\_start,x\_end,y\_end);**

■ **}**

■

■



**还能改进吗? ?**

## **void DDA\_circle(int n ,int x0,int y0,int radius)**

**// 把圆分成n等分, (x0,y0)为圆心坐标, radius为圆的半径**

- **{int i;**

- **Int x\_end,y\_end;     //线段的中点坐标**

- **my\_Moveto (x0+radius, y0)**

- **for(i=1;i<=n;i++)**

- **//循环地算终点坐标并连接这两点**

- **{**

```
x_end=x0+radius*cos(i*2*3.14/n) ;
```

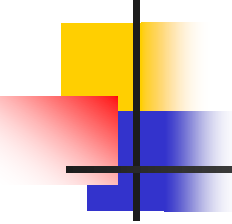
```
y_end=y0+radius*sin(i*2*3.14/n) ;
```

```
my_lineto(x_end,y_end) ;
```

- **}**

- **}**





# 直线Bresenham算法

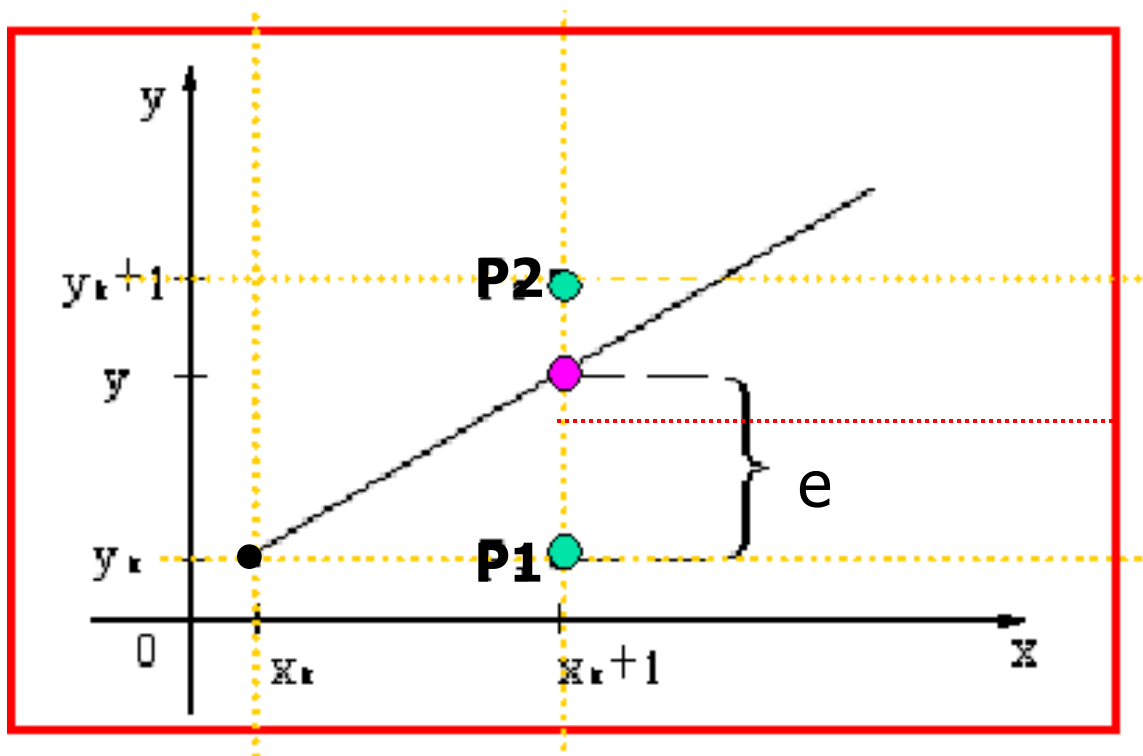
---

- **特点:**
  - -使用最广泛
  - -由误差项符号决定下一个像素取正右方像素还是右上方像素
- **优点**
  - 整数运算，速度快
  - 精度高

## ■ 基本思想

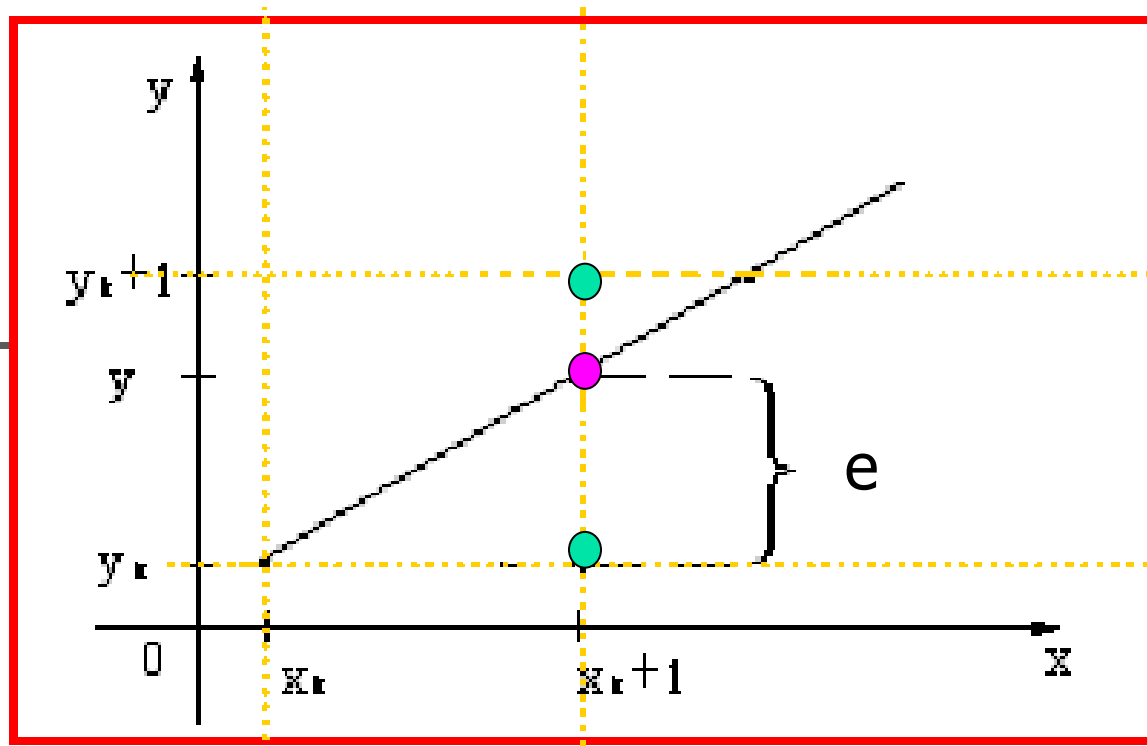
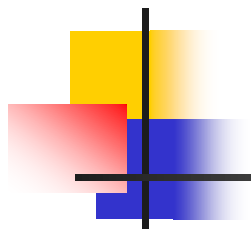
根据理想直线到位于直线下方的像素的距离 $e$ , 确定与理想直线最近的像素

以 $K \leq 1$ 为例

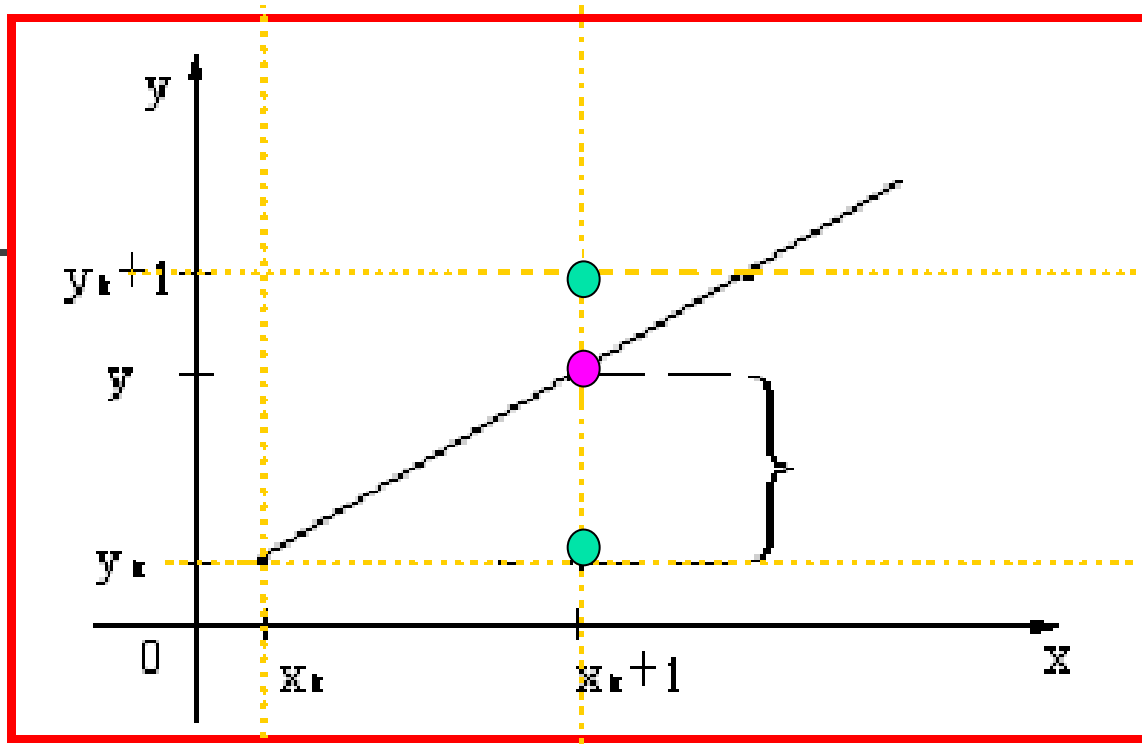


$e > 0.5$

$e > 0.5$



- **X方向: 每次走一步**  $x_{i+1}=x_i+1$
- **y方向: 走步与否**取决于误差**e**值的大小
- **如果  $e \geq 0.5$  时**, 最接近**P2( $x_i+1, y_i+1$ )** **y**方向走一步
- **如果  $e < 0.5$  时**, 最接近**P1( $x_i+1, y_i$ )** **y**方向不走步



- 误差 $e$ 计算

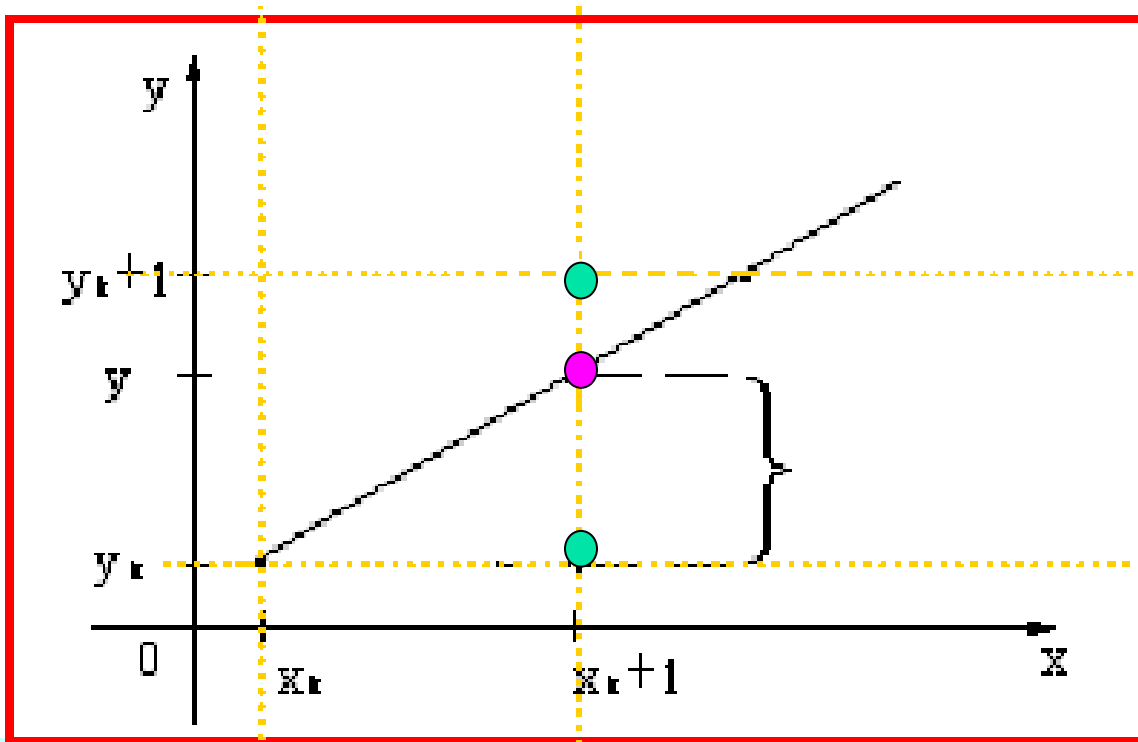
- 从第一点开始:

- $e = \Delta y / \Delta x$

■ 为方便与0比较，设

$$■ S = e - 0.5$$

$$■ S = \Delta y / \Delta x - 0.5$$



■ 当  $s \geq 0$  时，最接近  $P2(x_i+1, y_i+1)$   $y$  方向走一步

当  $s < 0$  时，最接近  $P1(x_i+1, y_i)$   $y$  方向不走步

■ 缺点：有除法，不宜硬件实现

改进 ■  $s = \Delta y / \Delta x - 0.5$   
 ■  $= (2\Delta y - \Delta x) / (2\Delta x)$   
 ■ 与0比较大小

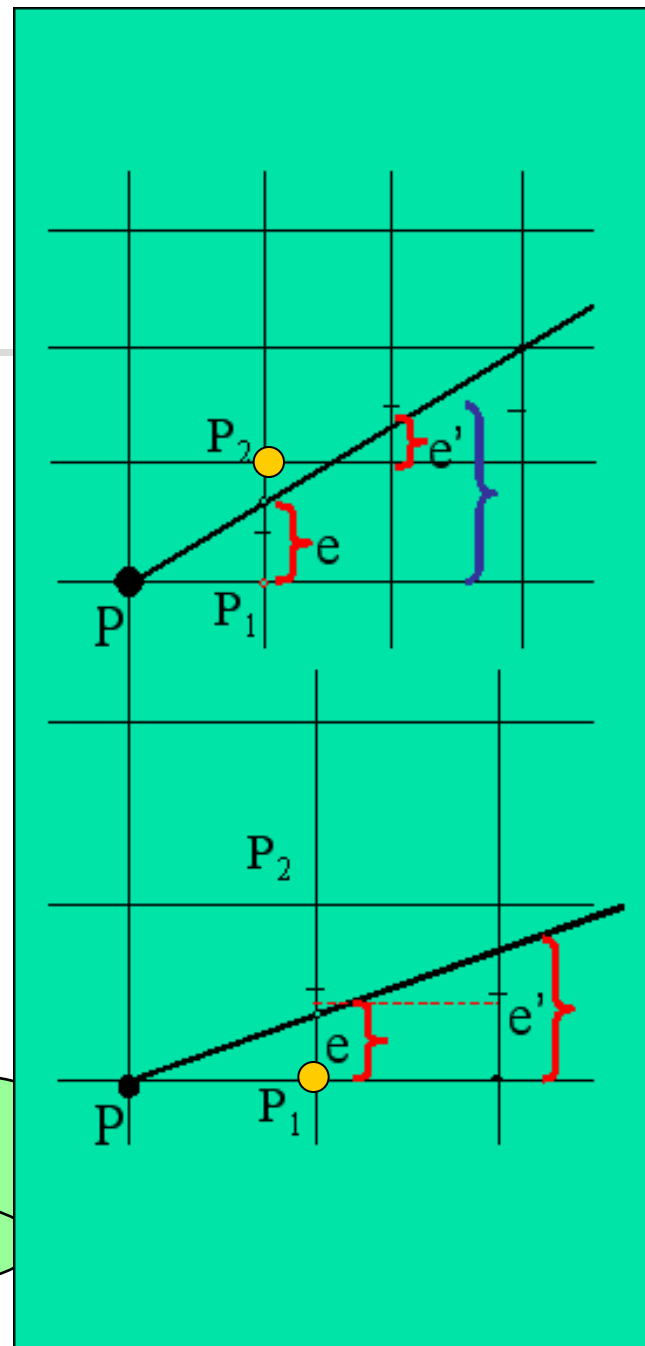
简化s与0的比较，拿分母与0比较

设  $t0 = 2\Delta y - \Delta x$

当  $t0 \geq 0$  时，  
 最接近  $P2(x_i+1, y_i+1)$   
 y方向走一步

当  $t0 < 0$  时，  
 最接近  $P1(x_i, y_i)$   
 y方向不走步

下一个点如何判断呢？



## ■ 上一步误差

$$■ e = \Delta y / \Delta x$$

## ■ 计算下一步误差的计算

■ 当  $t_0 \geq 0$  时，y 方向走一步

$$e' = 2e - 1 = 2\Delta y / \Delta x - 1$$

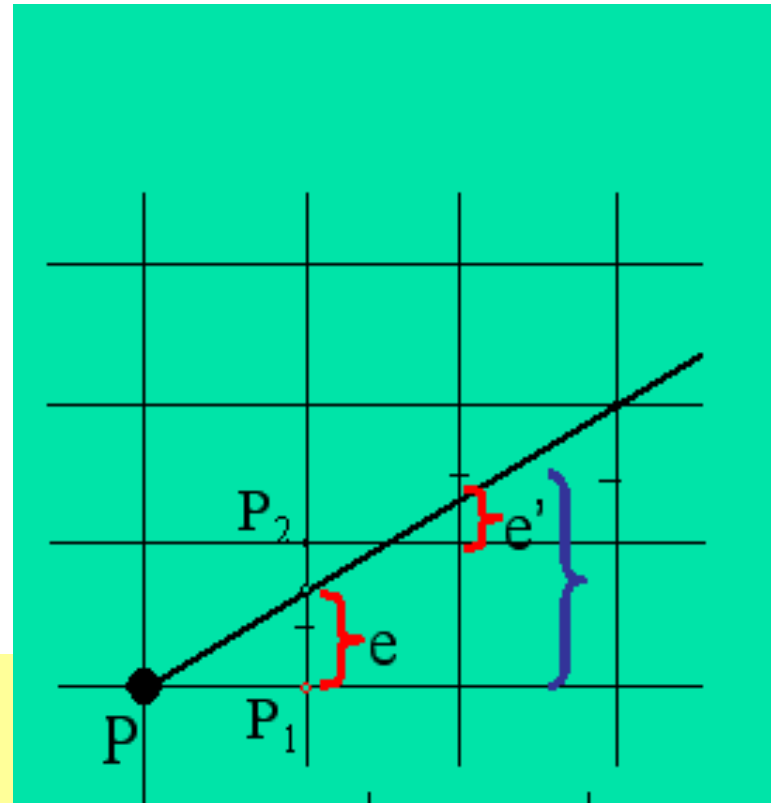
■ 同样为了方便与0比较

$$■ e' - 0.5 = 2\Delta y / \Delta x - 1 - 0.5$$

$$■ = (2\Delta y - \Delta x + 2\Delta y - 2\Delta x) / (2\Delta x)$$

■ 令分母为  $t_1 = t_0 + 2\Delta y - 2\Delta x$

$$\text{设 } t_0 = 2\Delta y - \Delta x \geq 0$$



- 当 $t_0 < 0$ 时，y方向不走步

设  $t_0 = 2\Delta y - \Delta x < 0$

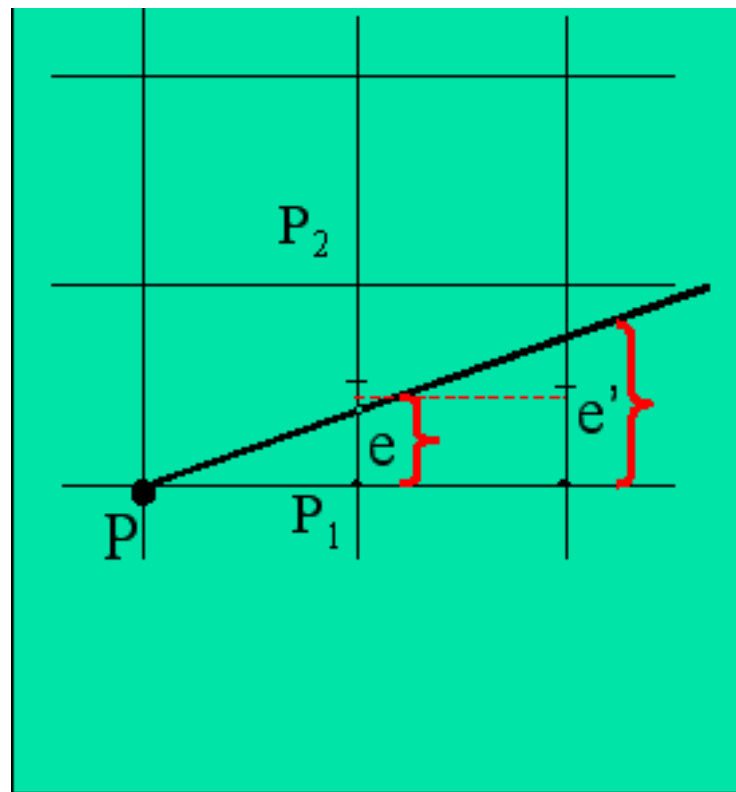
■  $e' = 2e = 2\Delta y / \Delta x$

- 同样为了方便与0比较

■  $e' - 0.5 = 2\Delta y / \Delta x - 0.5$

$= (2\Delta y - \Delta x + 2\Delta y) / (2\Delta x)$

令分母为  $t_1 = t_0 + 2\Delta y$





- 综上  $\Delta x=1$   $\Delta y=k$
- $t_0=2\Delta y - \Delta x=2K-1$

当  $t_i \geq 0$  时, y 方向走一步

$$\begin{aligned} t_{i+1} &= t_i + 2\Delta y - 2\Delta x \\ &= t_i + 2k - 2 \end{aligned}$$

当  $t_i < 0$  时, y 方向不走步

$$\begin{aligned} t_{i+1} &= t_i + 2\Delta y \\ &= t_i + 2k \end{aligned}$$

- **例：** 利用**Bresenham**算法画出**p0(0,0)**和**p'(6,2)**之间的一条直线。

**解答**  $K=(2-0)/(6-0)=1/3 \leq 1$

- **p0(0,0)**

$$t_0 = 2k - 1$$

$$\text{当 } t_i \geq 0 \text{ 时 } t_{i+1} = t_i + 2k - 2$$

$$\text{■ 当 } t_i < 0 \text{ 时 } t_{i+1} = t_i + 2k$$

• 起点**(0,0)**

**t<sub>i</sub>**

**坐标**

$$t_0 = 2k - 1 = -1/3 < 0$$

(1,0)

$$\because t_0 < 0 \therefore t_1 = t_0 + 2k = 1/3 > 0$$

(2,1)

$$\because t_1 > 0 \therefore t_2 = t_1 + 2k - 2 = -1 < 0$$

(3,1)

$$\because t_2 < 0 \therefore t_3 = t_2 + 2k = -1/3 < 0$$

(4,1)

$$\because t_3 < 0 \therefore t_4 = t_3 + 2k = 1/3 > 0$$

(5,2)

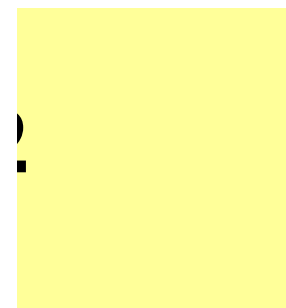
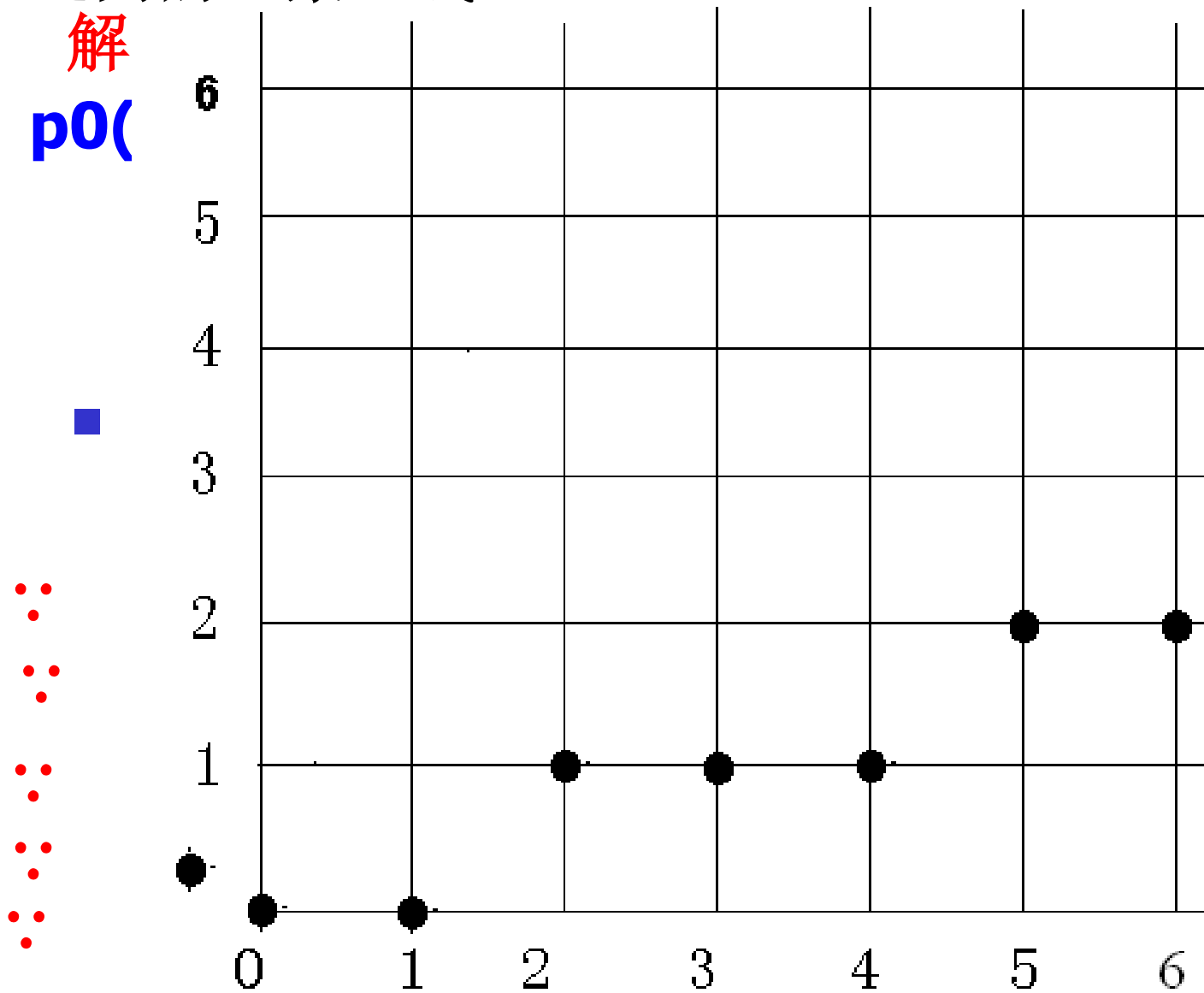
$$\because t_4 > 0 \therefore t_5 = t_4 + 2k - 2 = -1 < 0$$

(6,2)

- **例：** 利用 **Bresenham** 算法画出  $p_0(0,0)$  和  $p'(6,2)$  之间的一条直线。

**解**

- $p_0($



$p_1($

$)$

$)$

$)$

$)$

$)$

$)$

**例：**利用Bresenham算法画出 $p_0(3, 3)$ 和 $p'(7, 9)$ 之间的一条直线。 **解答**  $K=(9-3)/(7-3)=6/5 \geq 1$

$$1/k=4/6$$

$p_0(3,3)$

$$t_0 = 2(1/k) - 1 = 2/6$$

$$\text{当 } t_i \geq 0 \text{ 时 } t_{i+1} = t_i + 2(1/k) - 2$$

$$\text{■ 当 } t_i < 0 \text{ 时 } t_{i+1} = t_i + 2(1/k)$$

· 起点(3,3)

|     | $t_i$                                                             | 坐标    |
|-----|-------------------------------------------------------------------|-------|
| ■   | $t_0 = 2(1/k) - 1 = 2/6 \geq 0$                                   | (4,4) |
| ■ ∴ | $t_0 \geq 0 \quad \therefore t_1 = t_0 + 2(1/k) - 2 = -2/6 < 0$   | (4,5) |
| ■ ∴ | $t_1 < 0 \quad \therefore t_2 = t_1 + 2(1/k) = 1 \geq 0$          | (5,6) |
| ■ ∴ | $t_2 \geq 0 \quad \therefore t_3 = t_2 + 2(1/k) - 2 = 2/6 \geq 0$ | (6,7) |
| ■ ∴ | $t_3 \geq 0 \quad \therefore t_4 = t_3 + 2(1/k) - 2 = -2/6 < 0$   | (6,8) |
| ■ ∴ | $t_4 < 0 \quad \therefore t_5 = t_4 + 2(1/k) = 10 \geq 0$         | (7,9) |